Introduction to Programming

- **Definition and Purpose**:
  - Programming involves writing instructions that a computer can execute to perform specific tasks.
  - Used in applications ranging from web development to machine learning and scientific computing.
- **Types of Programming Languages**:
  - Low-level: Assembly, Machine Code (directly interacts with hardware).
  - High-level: Python, Java, C++ (user-friendly, abstracted from hardware).

Basic Concepts

- **Syntax and Semantics**:
  - Syntax: Rules defining the structure of valid code (e.g., indentation in Python).
  - Semantics: The meaning of a program or code block (e.g., what a function does).
- **Interpretation in Python**:
  - Python is an interpreted language, where source code is executed line by line by the Python interpreter.

Development Tools

- **IDEs**:
  - Examples: PyCharm, Visual Studio Code, Jupyter Notebook.
  - Features: Code editing, syntax highlighting, debugging, and version control.

Data Types

Python has five standard data types:

Primitive:

1. Number

```
# Number Examples
```

- int_num = 10 # Integer
- float_num = 20.5 # Float
- complex_num = 3 + 4j # Complex
- bool_val = True # Boolean

2. String- Group of Characters

Derived:

3. List (Mutable-[])
4. Tuple (Immutable-())
5. Dictionary-{ }

Derived Data Types

- **Lists**:
  - Dynamic arrays that can hold multiple data types.
  - Example: my_list = [1, 2, 3, "Hello"].
- **Tuples**:
  - Immutable sequences.
  - Example: my_tuple = (1, 2, 3).
- **Dictionaries**:
  - Key-value pairs.
  - Example: my_dict = {"key1": "value1", "key2": 10}.
- **Sets**:
  - Unordered collections of unique elements.
  - Example: my_set = {1, 2, 3}.

Flow of Control and Simple Functions

Control Flow Structures

- **Conditional Statements**:
  - if, elif, and else blocks to execute code based on conditions.
  - Example:

```
if x > 0:
print("Positive")
elif x == 0:
print("Zero")
else: print("Negative")
```

- **Looping Constructs**:
  - for loop: Iterates over sequences.

```
for i in range(5):
print(i)
```

  - while loop: Executes as long as a condition is true.

```
while x > 0:
print(x) x -= 1
```

Assignment: Student grade System based on average marks for number of subjects

Functions

- **Definition**:
  - Syntax: def function_name(parameters): // body.
  - Example:

```
def add(a, b):
return a + b
```

- **Recursion**:
  - Example:

```
def factorial(n):
if n == 1:
return 1
else:
return n * factorial(n - 1)
```

Data Structures

- **Linear Data Structures**
  - Arrays, linked lists, stacks, and queues
- **Non-Linear Data Structures**
  - Trees and graphs
- **Searching and Sorting Algorithms**
  - Binary search, merge sort, quicksort, etc.

Linear Data Structures

- **Lists**:
  - Example: my_list = [1, 2, 3]
  - Operations: append(), remove(), pop().
- **Stacks (using lists)**:
  - Example:

    stack = []

    stack.append(10)

    stack.pop()
- **Queues (using collections.deque)**:
  - Example:
  - from collections import deque

    queue = deque()

    queue.append(10)

    queue.popleft()